

Компиляторы компиляторов. Диаграммы Вирта

Теория автоматов и формальных языков

Лекция №13

Структура лекции

- Интерпретаторы.
- Компиляторы компиляторов.
- Диаграммы Вирта.
- Преобразование грамматик к более простому виду.

Интерпретаторы

- **Интерпретатор** — программа (разновидность транслятора), выполняющая *интерпретацию*.
- **Интерпретация** — пооператорный (покомандный, построчный) анализ, обработка и тут же выполнение исходной программы или запроса (в отличие от компиляции, при которой программа транслируется без её выполнения).

Типы интерпретаторов

- **Простой интерпретатор** анализирует и тут же выполняет (собственно интерпретация) программу покомандно (или построчно), по мере поступления её исходного кода на вход интерпретатора.
- **Интерпретатор компилирующего типа** — это система из *компилятора*, переводящего исходный код программы в промежуточное представление, например, в *байт-код* или *p-код*, и собственно интерпретатора, который выполняет полученный промежуточный код (так называемая *виртуальная машина*).

Особенности интерпретаторов

- Многие микропроцессоры интерпретируют машинный код с помощью встроенных микропрограмм, а процессоры семейства x86, начиная с Pentium (например, на архитектуре Intel P6), во время исполнения машинного кода предварительно транслируют его во внутренний формат (в последовательность микроопераций).
- Некоторые интерпретаторы могут быть написаны на языке, который они и интерпретируют. И Могут компилировать и интерпретировать сами себя. (PyPy - википедия)
- Задача написания простого интерпретатора интересна с точки зрения понимания описываемого языка.

Алгоритм работы простого интерпретатора

1. прочитать инструкцию;
2. проанализировать инструкцию и определить соответствующие действия;
3. выполнить соответствующие действия;
4. если не достигнуто условие завершения программы, прочитать следующую инструкцию и перейти к пункту 2.

Достоинства и недостатки интерпретаторов

Достоинства

- *Переносимость* интерпретируемых программ.
- Более совершенные и наглядные средства диагностики ошибок в исходных кодах.
- Меньшие размеры кода по сравнению с машинным кодом, полученным после обычных компиляторов.

Недостатки

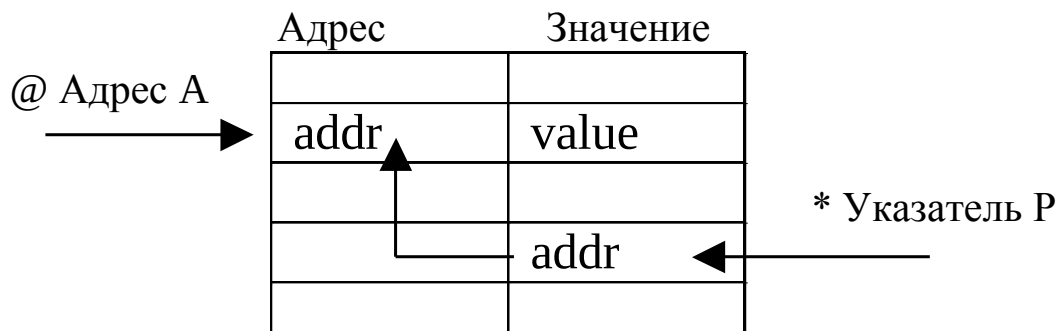
- Интерпретируемая программа не может выполняться отдельно без программы-интерпретатора.
- Интерпретируемая программа выполняется медленнее, поскольку промежуточный анализ исходного кода и планирование его выполнения требуют дополнительного времени.
- Практически отсутствует оптимизация кода.

Интерпретатор для польской формы записи

```
while TRUE do
begin
  case gettype(P[n]) of
    operand: Push(S,P[n]);
    operator: arg1 := Pop(); arg2 := Pop(); Push(arg1 @ arg2);
    else: error();
  endcase
  n := n+1
end
```


Типы аргументов

- константы,
- имена,
- адреса переменных (адрес значения, номер элемента таблицы),
- указатели (поле "Значение" содержит номер (адрес) элемента в таблице символов).



Неявные интерпретаторы

- Обычно интерпретатор – это самостоятельная программа, что зачастую создает некоторые проблемы, связанные с мобильностью программного обеспечения.
- Реализация неявного интерпретатора заключается в том, что формируется исполняемый файл, содержащий в себе как непосредственно интерпретатор, так и исходный текст программы (почти в явном виде).
(система программирования *Clipper*)

Brainfuck

- *Urban Müller*, <https://ru.wikipedia.org/wiki/Brainfuck>
- *Brainfuck* с бесконечным набором ячеек имеет тьюринговскую полноту, а, следовательно, по потенциальным возможностям не уступает языкам, подобным Си, Паскалю или Java.
- Машина, которой управляют команды Brainfuck, состоит из упорядоченного набора ячеек и указателя текущей ячейки, напоминая ленту и головку машины Тьюринга. Кроме того, подразумевается устройство общения с внешним миром (. и ,) через поток ввода и поток вывода.

Brainfuck

Команда Brainfuck	Эквивалент на Си	Описание команды
Начало программы	<pre>int i = 0; char arr[30000];</pre>	выделяется память под 30 000 ячеек
>	<pre>i++;</pre>	перейти к следующей ячейке
<	<pre>i--;</pre>	перейти к предыдущей ячейке
+	<pre>arr[i]++;</pre>	увеличить значение в текущей ячейке на 1
-	<pre>arr[i]--;</pre>	уменьшить значение в текущей ячейке на 1
.	<pre>putchar(arr[i]);</pre>	напечатать значение из текущей ячейки
,	<pre>arr[i] = getchar();</pre>	ввести извне значение и сохранить в текущей ячейке
[	<pre>while(arr[i]){</pre>	если значение текущей ячейки ноль, перейти вперёд по тексту программы на ячейку, следующую за соответствующей] (с учётом вложенности)
]	<pre>}</pre>	если значение текущей ячейки не ноль, перейти назад по тексту программы на символ [(с учётом вложенности)

“Hello, World!”

```
+++++[>++++>++++>++++>++++>+<<<<-]>+  
.>+.+++++..+++.>+.<<+++++>.>+.+.-----.>+.>.
```

Получение кодов букв и их вывод:

>++. Получение кода «Н» (72) из 70 в ячейке 1 и вывод «Н».

>+. Получение кода «е» (101) из 100 в ячейке 2 и вывод «е».

Malbolge

- Malbolge — эзотерический язык программирования, придуманный Беном Олмстедом в 1998 году.
- Язык разработан с целью быть максимально сложным для написания программ. Получил своё название от Malebolge, восьмого круга ада Данте.
- Потребовалось два года для написания первой программы.

Компиляторы компиляторов

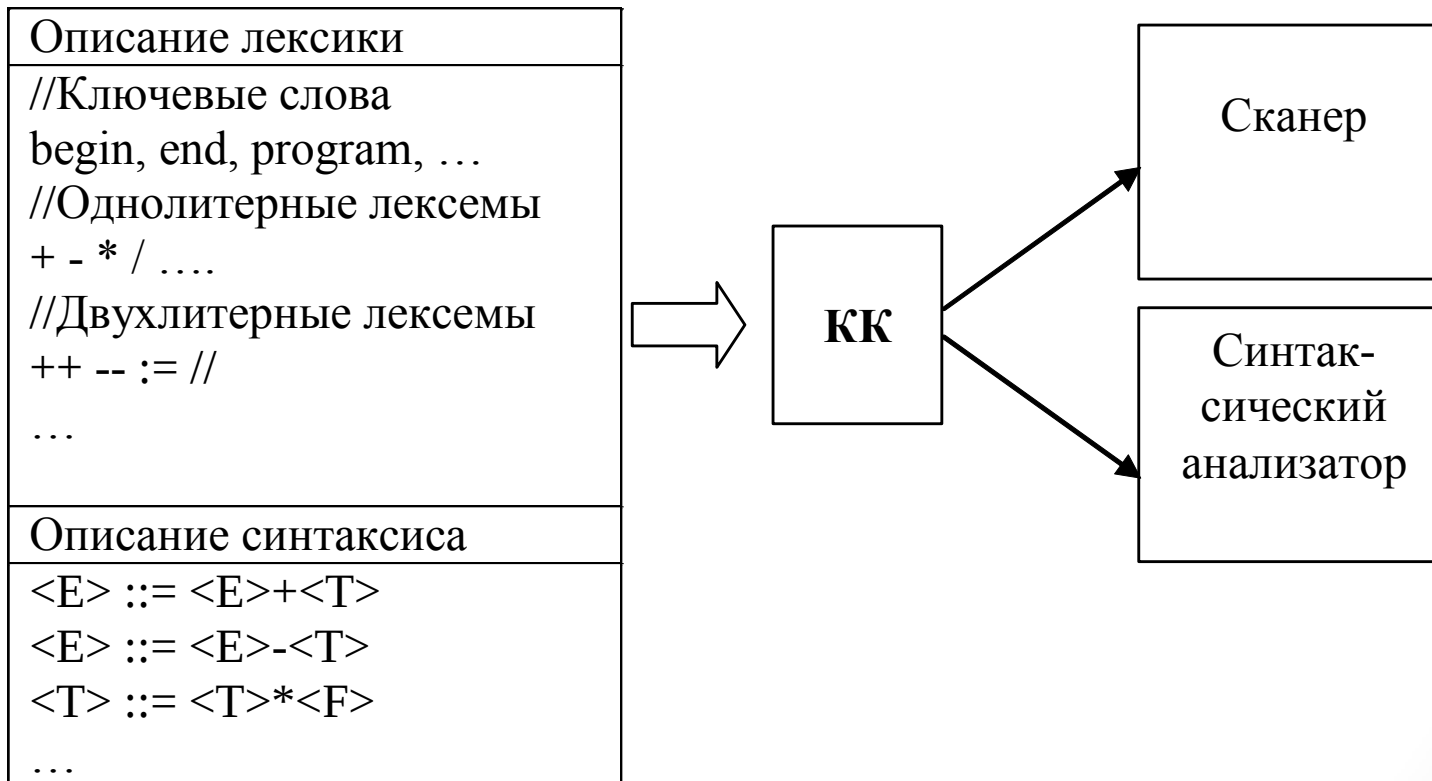
- **Компилятор компилятора (КК)** – это некий инструментарий программиста, помогающий создавать компиляторы (или интерпретаторы).
- Все рассмотренные методы анализа по возможности представлялись в формальном виде.
- Имея **регулярную грамматику** лексической структуры можно, *автоматически* сгенерировать сканер в виде конечного автомата.
- Имея **грамматику синтаксической структуры**, можно написать *универсальную* процедуру синтаксически управляемого перевода (или создать *универсальный* МП-автомат).

Компиляторы компиляторов

- Для генерации сканера конечного автомата потребуется некоторое описание двух грамматик – грамматики для сканера и грамматики для реализации синтаксического анализатора.
- Это описание вполне можно хранить в некотором файле.
- Тогда этот файл будет своего рода специальным входным языком – языком описания компилятора.

Язык описания компилятора

Описание компилятора



Компиляторы компиляторов

- При описании синтаксиса желательно правила грамматики записывать как можно в более естественной форме, при этом отделяя синтаксические категории от терминальных символов.
- В схеме СУ-перевода есть еще две составляющих – условия применимости правила грамматики и семантической процедуры.

Диаграммы Вирта

- Диаграммы Вирта позволяют достаточно просто решать ряд практических проблем, связанных с построением распознавателей, выполнять манипуляции с графами, а не с символьными преобразованиями.

Диаграммы Вирта

1. Избавление от левых или правых рекурсий.
2. Преобразование исходных синтаксических правил к более простому виду.
3. Преобразование $KC(n)$ грамматик к грамматикам с просмотром вперед на меньшее число символов, включая и преобразование к $KC(1)$ грамматике.
4. Проверка на эквивалентность различных грамматик.
5. Освобождение от пустых правил (сквозных связей).
6. Непосредственное использование правил с левой рекурсией.
7. Использование синтаксиса для определения семантики.
8. Доказательство принадлежности заданной грамматики к $KC(1)$ грамматике.

Избавление от левых и правых рекурсий

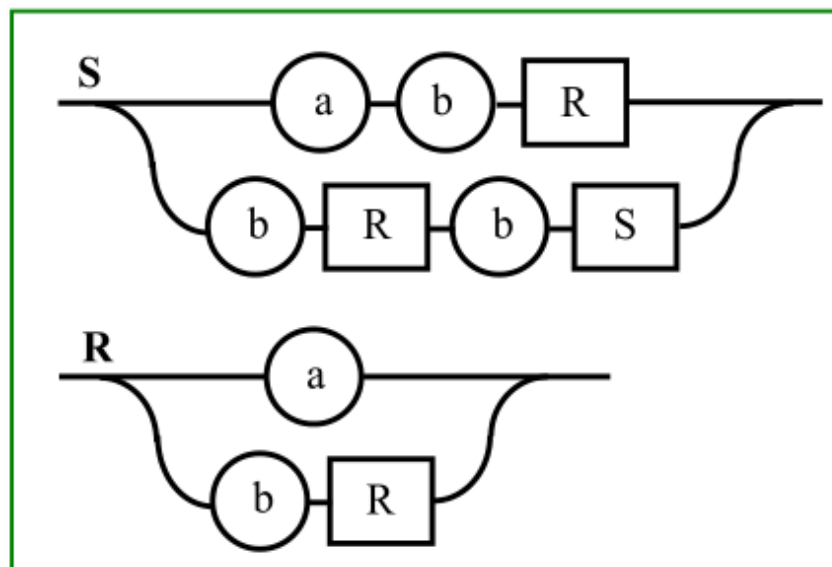
- При разработке нисходящих распознавателей сведение левой или правой рекурсии к итерации позволяет уменьшить вложенность правил.
- Наличие левой рекурсии не позволяет осуществлять нисходящий разбор.
- Поэтому, ее преобразование к итерации – это один из наиболее простых и удобных способов получения из грамматик, ориентированных на восходящий разбор, грамматик, используемых при нисходящем разборе.

Преобразование исходных данных синтаксических правил к более простому виду

- Диаграммы Вирта позволяют упростить исходную грамматику, что дает возможность перейти к более простым методам разбора.
- Рассмотрим **S-грамматику G**:

1. $S \rightarrow abR$
2. $S \rightarrow bRbS$
3. $R \rightarrow a$
4. $R \rightarrow bR$

Шаг 1



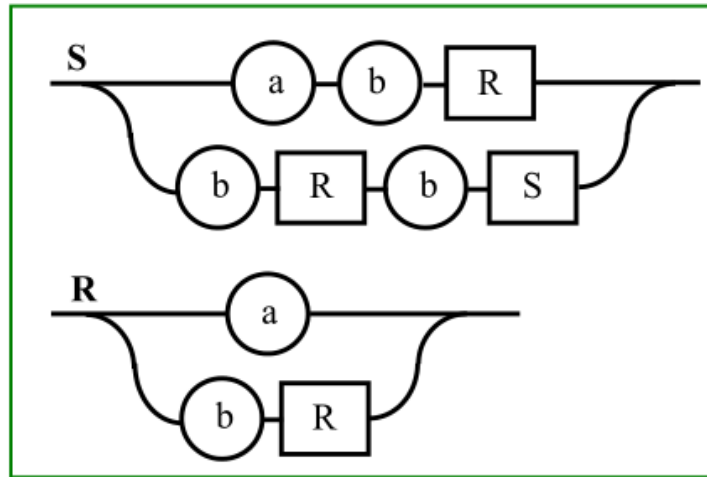
Шаг 2



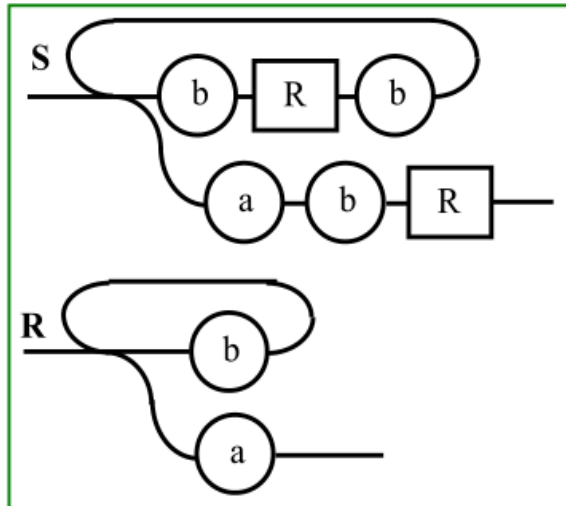
Преобразование исходных данных синтаксических правил к более простому виду

1. $S \rightarrow abR$
2. $S \rightarrow bRbS$
3. $R \rightarrow a$
4. $R \rightarrow bR$

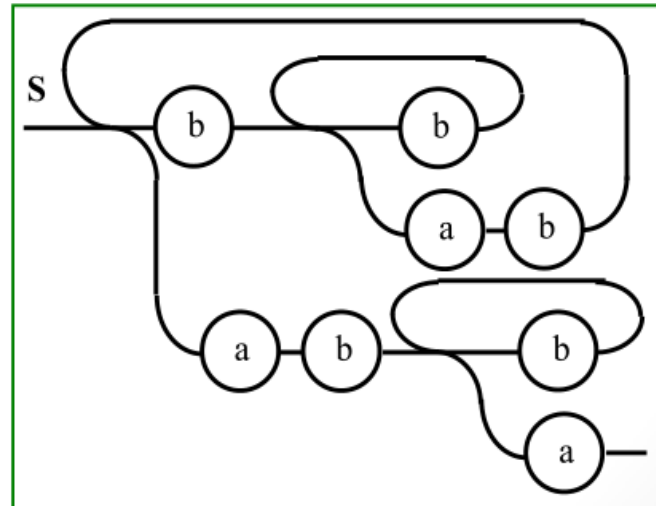
Шаг 1



Шаг 2



Шаг 3



Преобразование исходных данных синтаксических правил к более простому виду

